

OptiCode: An AI-Powered Intelligent Code Analysis Platform for Enhancing Computer Science Education

Mr. Haritha Himansha

*Dept. of Information Technology
Sri Lanka Institute of Information
Technology*

Malabe, Sri Lanka
it22601360@my.sliit.lk

Mr. Thilina Sandamal

*Dept. of Information Technology
Sri Lanka Institute of Information
Technology*

Malabe, Sri Lanka
it22606860@my.sliit.lk

Ms. Sanjeevi Chandrasiri

*Dept. of Information Technology
Sri Lanka Institute of Information
Technology*

Malabe, Sri Lanka
sanji.c@sliit.lk

Mr. Navinda Viraj

*Dept. of Information Technology
Sri Lanka Institute of Information
Technology*

Malabe, Sri Lanka
it22639226@my.sliit.lk

Ms. Ashani Hettiarachchi

*Dept. of Information Technology
Sri Lanka Institute of Information
Technology*

Malabe, Sri Lanka
it22604194@my.sliit.lk

Ms. Lokesh Weerasinghe

*Dept. of Information Technology
Sri Lanka Institute of Information
Technology*

Malabe, Sri Lanka
lokesha.w@sliit.lk

Abstract— The increasing reliance of computer science students on AI-generated code without adequate understanding of underlying concepts presents a significant pedagogical challenge. Students often produce functionally correct solutions while lacking the ability to explain algorithms, data structures, or design patterns, leaving them underprepared for technical assessments. This paper presents OptiCode, a multi-component intelligent code analysis platform that integrates four AI-powered modules: (1) a Code Concept Extractor using a hybrid Abstract Syntax Tree (AST) and Large Language Model (LLM) pipeline; (2) an Adaptive Weakness Detector that classifies recurring conceptual gaps; (3) a Code Refactoring and Risk Analyzer for structural improvement and risk-aware transformation; and (4) an AI Interview Simulator integrating multimodal performance evaluation through real-time facial emotion recognition and NLP scoring. Experimental evaluation across all four components demonstrates that: the hybrid AST-LLM Concept Extractor achieves a macro F1-score of 0.85, outperforming the LLM-only baseline by 16.4%; the hybrid Weakness Detector achieves $F1 = 0.92$; the Refactoring and Risk Analyzer achieves 90.7% user satisfaction in a study with 43 undergraduate students; and the Mini-Xception emotion recognition model achieves 70.4% accuracy at 18 ms inference latency. The integrated platform provides educators and learners with actionable, real-time feedback on code comprehension and technical readiness.

Keywords—code concept extraction, abstract syntax tree, large language model, adaptive learning, code refactoring, interview simulation, emotion recognition, computer science education

I. INTRODUCTION

The proliferation of AI code generation tools such as GitHub Copilot and ChatGPT has fundamentally altered how students interact with programming assignments. A substantial proportion of undergraduate computer science students now submit AI-generated solutions without developing an understanding of the embedded algorithmic concepts [1]. This behavior produces graduates who can write

syntactically correct programs yet struggle to explain time complexity, identify design patterns, or justify architectural decisions during technical interviews.

Traditional learning platforms such as LeetCode assess functional correctness but not conceptual understanding. Static linters detect syntactic issues without identifying higher-order concepts such as Recursion or the Observer Pattern. This gap motivates a platform that understands code semantically and provides concept-level feedback.

This paper presents OptiCode, a microservice-based platform combining four AI-driven components. The system pursues four objectives: (i) automatically extract and explain CS concepts in student code; (ii) identify and track conceptual weaknesses over time; (iii) improve code quality through intelligent refactoring; and (iv) simulate technical interviews with multimodal performance scoring.

II. RELATED WORK

A. Automated Code Analysis

Existing work on program comprehension focuses on code summarization [2] and comment generation [3]. These approaches produce natural language descriptions but do not map constructs to named CS concepts. Recent LLM-based methods have demonstrated strong code understanding capabilities — notably through models trained on large code corpora [4] and general-purpose models such as GPT-4 [11] — but remain prone to hallucination and ignore structural precision. Dakhel et al. [12] evaluated GitHub Copilot as an AI pair programmer and found that while it reduces development time, it frequently generates syntactically correct but semantically flawed code, reinforcing the need for structural verification. Our hybrid pipeline addresses these limitations by grounding LLM inference with determinis

AST analysis, combining the semantic breadth of LLMs with the precision of static structural inspection.

B. Adaptive Learning Systems

Intelligent tutoring systems such as Cognitive Tutor [5] model student knowledge states but require extensive manual knowledge engineering. Neural knowledge tracing [6] predicts learning outcomes but lacks concept-level diagnostic feedback suitable for programming education. A systematic review by Messer et al. [13] found that most automated programming assessment systems rely primarily on unit testing, where feedback is limited to output correctness without diagnosing underlying conceptual errors. Recent adaptive systems have begun integrating LLMs for personalized hint generation [14], yet few combine real-time structural code analysis with progressive knowledge modelling, which is the approach adopted in the proposed Adaptive Weakness Detector.

C. Code Refactoring and Interview Simulation

AST-based refactoring tools are well-established in IDEs [7] but do not provide educational rationale or risk scoring. Prior interview simulation research addresses speech analysis [8] and text-based responses [9] but rarely combines real-time facial affect recognition with technical content evaluation. Recent work has begun integrating LLM-based dialogue management into interview systems [15], yet these approaches still lack continuous behavioral monitoring and adaptive questioning grounded in prior performance — both of which are central to the proposed AI Interview Simulator.

III. SYSTEM ARCHITECTURE

OptiCode is implemented as a distributed microservice architecture. A React/Vite frontend communicates with a Node.js/Express API gateway backed by MongoDB. Four specialized AI services, each exposing a REST API, handle domain-specific inference tasks. The gateway enforces JWT-based authentication and rate limiting. Table I summarizes the microservice layout.

TABLE I OPTICODE MICROSERVICE SUMMARY

Service	Function	Tech	Port
Concept Extractor	AST+LLM detection	FastAPI	8000
Weakness Detector	Gap classification	Flask	8001
Risk Analyser	Refactoring & risk	Flask	8002
Interview Sim.	Emotion+NLP score	Flask	8003

IV. COMPONENT 1: CODE CONCEPT EXTRACTOR

A. Motivation

Students frequently submit syntactically correct code without demonstrating understanding of the underlying computer science concepts. Existing tools primarily evaluate correctness rather than conceptual comprehension. The proposed Concept Extractor addresses this gap by identifying

and explaining high-level concepts such as algorithms, data structures, and design patterns present in source code.

B. Hybrid AST + LLM Pipeline

The extractor implements a three-stage hybrid pipeline combining deterministic static analysis with semantic inference.

Stage 1—Static AST Analysis. Source code is parsed using Python’s ast module performs structural analysis: it identifies classes, functions, inheritance chains, recursive calls, design pattern signatures (Singleton, Observer, Factory), and control flow constructs. This produces a pre-detected patterns dictionary with high precision but limited semantic recall.

Stage 2—LLM-Augmented Extraction. Structural findings are injected as confirmed context into a prompt sent to Google Gemini 2.0 Flash via the Generative Language API. The hybrid prompt instructs the model to include all AST-confirmed concepts and to supplement them with additional semantic observations. The LLM-only baseline receives raw code with no structural hints and is limited to five high-confidence concepts, modelling the conservative behavior of an unaided language model.

Stage 3—Post-Merge and Confidence Fusion. Concepts confirmed by both the LLM and AST receive a confidence boost of +0.10. AST-detected concepts absent from the LLM output are appended with source tag ast_analysis. This recovers false negatives that the LLM omits due to its conservative stance.

C. Confidence Scoring

Each concept carries a confidence score $c \in [0, 1]$. AST-only detections: $c = 0.72$, a value determined empirically as the mean precision of AST-only extractions observed across a held-out validation subset of 20 samples prior to full evaluation; this reflects the inherent recall-precision trade-off of deterministic pattern matching, which is high in precision but occasionally overfires on partial structural matches. LLM-only detections use the model’s self-reported score directly. For hybrid detections where both the AST and LLM independently confirm the same concept, a confidence boost of +0.10 is applied: $c = \min(1.0, c_{\text{LLM}} + 0.10)$. This boost was determined empirically by comparing agreement-rate statistics across the validation subset, where dual-source agreement consistently corresponded to a ~10 percentage point improvement in ground-truth correctness relative to LLM-only confidence. This calibrated scoring enables reliable uncertainty estimation across all extraction modes.

V. COMPONENT 2: ADAPTIVE WEAKNESS DETECTOR

A. Motivation

A single extraction reveals which concepts a code sample contains; it does not reveal which concepts a student consistently misunderstands. The Weakness Detector analyses extraction histories over multiple sessions to identify recurring gaps. However, many existing adaptive programming learning systems rely on coarse-grained indicators such as aggregate performance or predefined learning tracks. Furthermore, a systematic review by Messer et al. shows that most automated assessment systems rely primarily on unit testing, where feedback is limited to correctness of output without diagnosing underlying conceptual errors.

To address these limitations, the proposed system introduces a unified adaptive framework that integrates real-time weakness detection, targeted task generation, and performance-driven progression, enabling continuous and fine-grained personalization for Python programming education.

B. Approach

Concept extraction records are accumulated per user in MongoDB. A feature vector from concept frequency and confidence trends is used by a machine learning classifier to estimate the likelihood of each concept representing a persistent weakness. The system generates personalized study recommendations targeting the weakest areas, with a target response latency under 3 seconds.

Stage 1— Initial Skill Assessment and Student Placement. At the start, students complete a short assessment with coding and conceptual questions. A Random Forest classifier predicts the learner’s level (Beginner, Intermediate, Advanced), which is treated as a dynamic state and updated based on ongoing performance.

Stage 2— Hybrid Weakness Detection. During task execution, the system performs real-time analysis using a hybrid model combining AST-based structural features, execution output comparison, rule-based checks (e.g., hardcoded values, missing outputs), and behavioral signals such as idle time. Detected weaknesses are categorized and used to generate immediate, personalized feedback.

Stage 3— Weakness-Aware Task Generation. Tasks are dynamically selected based on detected weaknesses rather than a fixed sequence. The system supports both progression-based tasks within the current level and focused practice tasks targeting specific weak concepts.

Stage 4— Knowledge Modelling using BKT. Learner mastery is modelled using Bayesian Knowledge Tracing (BKT) [10], where mastery probability is updated after each interaction. Level-dependent learning rates (0.30, 0.25, 0.20) are applied to reflect varying learning dynamics.

Stage 5—Multi-Factor Level Progression. Level progression is determined using a combination of mastery (≥ 0.85), task completion thresholds (e.g., 10 tasks for

Beginner), and performance indicators such as accuracy and solving time.

Stage 6—Engagement and XP Mechanism. An XP-based system rewards students based on weakness, severity and solving efficiency. Repeated weaknesses trigger targeted practice to ensure continuous improvement.

VI. COMPONENT 3: CODE REFACTORING AND RISK ANALYZER

A. Motivation

Students often write functionally correct code that are structurally suboptimal, leading to poor maintainability and increased technical debt. Although existing static analysis tools and IDE linters can detect syntactic issues and style violations, they provide limited support in explaining refactoring decisions or quantifying the risk associated with code transformations. As a result, learners rarely develop a deep understanding of why certain code patterns are harmful or how structural improvements impact software quality.

The Code Refactoring and Risk Analyzer addresses this gap by identifying refactoring opportunities through AST-based analysis while quantifying the associated transformation risk. To enhance learning effectiveness, the component integrates an AI-driven explanation module that provides rationale for each refactoring decision and its impact on code quality. Furthermore, a gamified analytics dashboard visualizes developer progress using metrics such as cyclomatic complexity, maintainability index, and risk distribution trends, thereby enabling continuous, measurable improvement toward production-grade software development practices.

B. Approach

The component parses submitted code using language-specific ASTs and applies a rule-based transformation engine. Candidate refactorings extract method, replace magic number, simplify conditional—are evaluated against a risk model considering scope of change, test coverage indicators, and coupling metrics. Transformations are classified as low, medium, or high risk and presented with an educational rationale. The deterministic AST approach ensures 100% of suggested transformations produce valid output.

Stage 1—Static AST Analysis. Before sending code to any neural network, the system utilizes AST parsing to decompose the program into its structural components—nodes, branches, and expressions. This pre-processing step provides the AI models with structured input that captures the logical organization of the program rather than its surface-level syntax and produces a pre-detected patterns dictionary with high precision.

Stage 2—Code Smell Detection. By traversing the AST, the system accurately detects deep-rooted anti-patterns such as deeply nested loops, high cyclomatic complexity, dead code, and bloated classes or methods. These structural

issues are flagged before any AI inference, ensuring deterministic and verifiable detection.

Stage 3—Safe Transformation. The system ensures behavioral preservation by comparing the ASTs of the original and refactored code, maintaining correctness while improving maintainability. Candidate refactoring extract method, replace magic number, simplify conditional—are evaluated against a risk model considering scope of change, test coverage indicators, and coupling metrics. The deterministic AST approach ensures 100%

Stage 4—Risk Classification: Each candidate transformation is assigned a risk level of low, medium, or high based on the scope of change, coupling metrics, and test coverage indicators. Classifications are presented alongside an educational rationale so that students understand not only what was changed but why the transformation carries a given level of risk, reinforcing safe coding and refactoring practices.

VII. COMPONENT 4: AI INTERVIEW SIMULATOR

A. Motivation

Technical interviews assess both answer correctness and non-verbal communication. Students rarely practice under simulated conditions, and existing mock platforms provide only text-based scoring.

Existing interview systems primarily rely on static evaluation and do not adequately support real-time conversational interaction or behavioral analysis. Although recent AI-based approaches introduce speech-based interaction using text-to-speech (TTS) and large language models (LLMs), they still lack integrated assessment of both technical responses and non-verbal behavior.

To address this gap, there is a need for an intelligent interview system that enables real-time speech interaction, continuous facial emotion analysis, and adaptive questioning based on candidate performance. In addition, the system incorporates personalized follow-up questioning based on previously completed tasks and responses, enabling deeper assessment of candidate understanding.

This capability is supported through an Adaptive Weakness Detection module, which identifies performance gaps from prior answers and dynamically generates targeted questions to address those weaknesses. This facilitates a more realistic interview simulation in which both technical competence and behavioral dynamics are evaluated simultaneously.

B. Approach

The simulator presents a randomized technical question from a curriculum-aligned question bank. A webcam feed is processed in real time using OpenCV and a Mini-Xception model to classify the student’s emotional state across seven affect categories: neutral, focused, confused, anxious, confident, distracted, and engaged. The student’s spoken or

typed response is evaluated by an NLP scoring module assessing technical accuracy, completeness, and terminology precision. The final score combines a content component and an affective profile component. The weighting of these components — content at 0.7 and affect at 0.3 — was determined through an empirical validation process during system development, in which three candidate distributions (0.6/0.4, 0.7/0.3, and 0.8/0.2) were evaluated against expert interviewer ratings collected across five mock interview sessions. The 0.7/0.3 split produced the highest correlation with expert judgements and is consistent with prior work in multimodal interview assessment [8], which identifies technical content as the primary determinant of hiring outcome while non-verbal behavior serves as a secondary signal. The final score is therefore computed as:

$$S_{final} = 0.7 \times S_{content} + 0.3 \times S_{affect}$$

Stage 1— Adaptive Questioning. Identifies candidate weaknesses from previous tasks and generates personalized questions using Gemini 2.5 Flash to enable adaptive interview progression.

Stage 2 — Real-Time Interaction. Supports live speech-based interview interaction using ASR (AssemblyAI), TTS (Cartesia), and Gemini 2.5 Flash for dialogue management and dynamic follow-up questioning.

Stage 3 — Emotion Analysis Performs real-time facial emotion recognition using OpenCV and a Mini-Xception model to track candidate engagement and behavioral state during the interview.

Stage 4 — Scoring & Feedback: Candidate responses are evaluated using an NLP module for technical accuracy, completeness, and terminology precision. An affective score is computed using the Mini-Xception facial emotion recognition model, which analyses real-time emotional states throughout the interview. Each detected emotion is assigned a predefined score reflecting its relevance to interview performance: positive states such as happy, confident, and engaged contribute positively, while negative states such as sad, fearful, or anxious contribute negatively. The affective score is computed as the temporal mean of these weighted emotion values across the interview session, representing 30% of the final evaluation. The system then generates structured feedback addressing both technical performance and behavioral indicators, further summarized using Gemini 2.5 Flash.

VIII. EXPERIMENTAL EVALUATION

A. Dataset and Protocol

1) *Code Concept Extractor Dataset:* Due to the absence of publicly available benchmark datasets for code concept extraction, a custom ground-truth dataset was constructed. The dataset consists of 120 Python code samples spanning

three difficulty levels (40 easy, 40 medium, 40 hard). Samples cover classical algorithms (binary search, merge sort, dynamic programming), data structures (linked list, stack, binary search tree), and design patterns (Singleton, Observer, Factory). Three extraction modes were evaluated: AST-only, LLM-only, and Hybrid. Performance was measured using precision, recall, and F1-score. Since concept labels may vary lexically (e.g., "DFS" vs. "Depth-First Search"), fuzzy string matching based on Levenshtein edit distance similarity was applied to compare predicted and ground-truth concept labels. A similarity threshold of 0.80 was selected as the matching cutoff; label pairs scoring below this threshold were treated as non-matching. This threshold was validated on a sample of 30 label pairs manually reviewed by two annotators, achieving inter-annotator agreement of 93%, confirming that 0.80 effectively captures semantically equivalent labels while excluding spurious partial matches. Expected Calibration Error (ECE) was computed by binning predictions into five confidence buckets.

2) *Adaptive Weakness Detector Dataset*: The Adaptive Weakness Detector was evaluated on a balanced labelled dataset of 600 Python code samples, comprising 100 samples per category across six classes: `syntax_error`, `missing_base_case`, `infinite_loop`, `logic_error`, `idle_stuck`, and `none`. Three additional categories — `hardcoded_value`, `missing_print`, and `no_function` — are handled by deterministic rule-based checks, giving the complete system eight detectable weakness types. Three modes were evaluated: Rule-based only, ML-only, and Hybrid. The hybrid mode applies rule-based checks as a priority filter before falling back to the trained ML classifier. Precision, recall, and F1-score were computed per category against ground-truth labels.

3) *Code Refactoring and Risk Analyzer Dataset*: A user study was conducted with 43 undergraduate computer science students at SLIIT across two laboratory sessions. Participants submitted their own Python programming assignments for refactoring analysis and interacted with the complete component pipeline—including the AST-based engine, risk classifier, chatbot, and analytical dashboard. Upon completion, participants responded to a five-point Likert-scale questionnaire (1 = Strongly Disagree, 5 = Strongly Agree) covering five evaluation criteria: refactoring accuracy, risk explanation clarity, dashboard usefulness, chatbot helpfulness, and overall satisfaction. A comparative evaluation against SonarQube and ChatGPT was additionally conducted across six capability dimensions to contextualize the system's performance relative to existing tools.

4) *AI Interview Simulator Dataset*: The emotion recognition module of the AI Interview Simulator is trained using the FER2013 dataset, which contains 35,887 grayscale

facial images of size 48×48 pixels, labeled across seven emotion categories (e.g., happy, sad, neutral, fear). The dataset is split into training, public test, and private test sets. A Mini-Xception CNN model is employed for facial emotion classification due to its efficiency and suitability for real-time inference. The images are preprocessed through normalization and resizing, and data augmentation is applied to improve robustness and generalization. The trained model is integrated into the system for real-time emotion detection using webcam input, and the predicted emotion labels are mapped into affect scores that contribute to the final interview evaluation. In addition, Gemini 2.5 Flash is utilized for response-level scoring interpretation and structured feedback generation. It analyzes both the NLP-based technical evaluation results and the affective behavior scores to produce human-readable feedback, highlighting candidate strengths, weaknesses, and improvement areas.

B. Results

1) Code Concept Extractor Results:

TABLE II CONCEPT EXTRACTION EVALUATION (120 SAMPLES)

Mode	Prec.	Recall	F1(Macro)	Std(F1)	ECE
AST-only	0.89	0.65	0.75	0.05	0.10
LLM-only	0.87	0.62	0.73	0.06	0.13
Hybrid	0.86	0.85	0.85	0.04	0.07

Table II shows that hybrid approach achieves a macro F1-score of 0.85 ± 0.04 , outperforming both AST-only and LLM-only baselines. Compared to the LLM-only configuration, the hybrid model improves F1 by approximately 16.4%, primarily due to a substantial increase in recall.

AST-only achieves the highest precision due to deterministic structural matching but exhibits lower recall as it cannot capture implicit semantic concepts. In contrast, the LLM-only approach demonstrates improved flexibility but suffers from conservative extraction behavior, leading to missed concepts. The hybrid model effectively combines both approaches, using AST-based structural grounding to ensure reliability while leveraging LLM-based inference to improve semantic coverage.

The hybrid configuration also achieves the lowest Expected Calibration Error (ECE = 0.07), indicating better alignment between predicted confidence scores and actual correctness

TABLE III SYSTEM CAPABILITY COMPARISON

Capability	LeetCode	ChatGPT	OptiCode
Named CS concept extraction	0/30	13/30	29/30
Confidence scoring per concept	0/30	0/30	28/30
Hallucination-free extraction	0/30	11/30	27/30

Concept-level explanation	0/30	17/30	28/30
Design pattern detection	0/30	14/30	27/30
AST-grounded structural analysis	0/30	6/30	26/30
Tailored to CS education	9/30	8/30	29/30
Consistent structured output	18/30	9/30	28/30

Across all evaluated capabilities, OptiCode consistently outperformed both LeetCode and ChatGPT in terms of educational and structural understanding of code. OptiCode achieved the highest user agreement across all metrics, ranging from 26/30 (86.7%) to 29/30 (96.7%), indicating strong consistency in delivering meaningful and pedagogically relevant outputs.

In comparison, ChatGPT demonstrated moderate performance in explanation-oriented tasks (up to 17/30, 56.7%) but showed limitations in structured concept extraction and confidence scoring. LeetCode, while useful for practice problems, showed minimal support for advanced conceptual interpretation, with most capabilities scoring near zero except for structured output familiarity (18/30, 60%).

Overall, the results validate that OptiCode provides a significantly more effective learning-oriented programming analysis experience compared to existing general-purpose and practice-based platforms.

2) Adaptive Weakness Detector Results:



Fig. 1. Mode comparison chart

As shown in “Fig. 1”, the hybrid system achieves F1 = 0.92, outperforming rule-based detection (F1 = 0.57) and remaining competitive with ML-only (F1 = 0.97) while providing more robust real-world performance through deterministic pre-filtering. Rule-based detection failed entirely on `logic_error` and `missing_base_case`, confirming that execution-based ML analysis is essential for these categories. `missing_base_case` recorded the lowest hybrid F1 (0.80) due to structural overlap with `idle_stuck` at the AST feature level. All remaining categories achieved $F1 \geq 0.86$.

3) Code Refactoring and Risk Analyzer Results:

TABLE IV COMPARISON WITH EXISTING REFACTORING TOOLS

Feature	SonarQube	ChatGPT	OptiCode (Proposed)
Automated Refactoring	Partial	Single-shot	Multi-model comparison
Structural Safety (AST)	High (rule-based)	Low-Medium (text-based)	High (AST + AI)
Tech Debt Tracking	Yes (accumulation)	None	Yes (reduction over time)
Risk Classification	Rule-based	Prompt-dependent	ML-driven (Critical-Low)
Educational Rationale	Docs only	Conversational	Contextual chatbot
Auto Test Generation	None	Manual prompt	Automatic

Table III compares OptiCode against SonarQube and ChatGPT across six capability dimensions central to educational refactoring. OptiCode outperforms both baselines across all six dimensions. While SonarQube achieves high structural safety through deterministic rules, it lacks the ability to perform automated refactoring or act as an interactive mentor. ChatGPT offers strong conversational capability but treats code as plain text, sacrificing structural safety and providing no persistent technical debt tracking. OptiCode's AST-grounded engine is the only system to address all six dimensions simultaneously.

TABLE V USER STUDY RESULTS CODE REFACTORING COMPONENT (N = 43)

Valuation Criterion	Mean (out of 5)	Std. Dev.	Satisfied (%)
Refactoring Accuracy	4.31	0.61	89.5%
Risk Explanation Clarity	4.18	0.74	86.0%
Dashboard Usefulness	4.42	0.58	91.6%
Chatbot Helpfulness	4.25	0.67	88.4%
Overall Satisfaction	4.36	0.55	90.7%

The dashboard received the highest satisfaction rating (mean 4.42, 91.6% satisfied), indicating that visual tracking of technical debt and developer growth was highly valued. Refactoring accuracy and chatbot helpfulness scored above 88% satisfaction. Risk explanation clarity received the lowest mean (4.18), suggesting further refinement of natural-language rationale for high-risk transformations is warranted. Overall, 90.7% of participants reported satisfaction, validating the component's effectiveness as an educational refactoring tool.

Each mean score was computed by averaging raw Likert responses (1 = Strongly Disagree, 5 = Strongly Agree) across all 43 participants per criterion. The standard deviation reflects the degree of agreement; lower values (e.g., 0.55 for Overall Satisfaction) indicate high consensus, while slightly higher values (e.g., 0.74 for Risk Explanation Clarity) reveal moderate variation. The satisfaction percentage was calculated as the proportion of respondents who selected 4 or 5 out of 43, expressed as a percentage. For example, for Dashboard Usefulness, 39 out of 43 students rated it 4 or above, yielding $(39/43) \times 100 = 90.7\%$.

4) AI Interview Simulator Results:

TABLE VI PERFORMANCE COMPARISON OF FACE EMOTION RECOGNITION MODELS COMPARISON

Model	Accuracy (%)	Precision(%)	Recall (%)	Latency (ms)
Mini-Xception	70.4	69.8	69.2	18
resnet50	66.4	65.8	65.0	85

Although ResNet50 is a deeper architecture, Mini-Xception achieves higher overall performance in this experimental setting, with improved accuracy (70.4% vs. 66.4%), precision (69.8% vs. 65.8%), and recall (69.2% vs. 65.0%). In addition, Mini-Xception provides significantly lower inference latency (18 ms compared to 85 ms), making it more suitable for real-time facial emotion recognition in the AI Interview Simulator.

This improvement is primarily attributed to Mini-Xception’s optimized lightweight design, which reduces computational overhead while maintaining strong feature extraction capability for FER2013-based emotion classification. In contrast, ResNet50 introduces higher computational complexity, which increases latency and negatively impacts real-time responsiveness despite its deeper representational capacity.

Therefore, Mini-Xception is selected as the final deployment model due to its superior trade-off between accuracy and real-time performance, which is critical for continuous webcam-based interview analysis.

C. Difficulty-Stratified Analysis

On easy samples (single-concept, short code), all modes perform comparably ($F1 \approx 0.85$). The hybrid advantage is most pronounced on hard samples (multi-pattern, 100+ LOC), where recall improves from 0.44 (LLM-only) to 0.71 (hybrid), confirming that AST structural grounding is most valuable for complex code where the LLM is most likely to under extract.

IX. DISCUSSION

The results support the central hypothesis: AST pre-detection significantly improves concept recall without sacrificing practical utility. The key mechanism is the post-

merge recovery of LLM false negatives by the AST. For simple code, the LLM alone is sufficient; for complex, multi-pattern code, the AST contribution is decisive, as evidenced by recall improving from 0.44 (LLM-only) to 0.71 (hybrid) on hard samples.

The precision reduction in hybrid mode (0.86 vs. 0.89) reflects AST false positives arising from partial pattern matches — for instance, detecting a Singleton-like structure that does not constitute a fully implemented pattern. Future work will apply stricter confidence thresholds and pattern validation rules to reduce such false positives without compromising recall.

The Adaptive Weakness Detector’s hybrid mode ($F1 = 0.92$) demonstrates that rule-based pre-filtering and ML-based inference are complementary rather than redundant. The lower F1 for missing_base_case (0.80) highlights a known limitation of AST-level feature overlaps with idle_stuck states, which will be addressed through richer behavioral feature engineering in future iterations.

The Code Refactoring and Risk Analyzer user study ($n = 43$) yielded strong overall satisfaction (90.7%), though risk explanation clarity received the lowest mean score (4.18), suggesting that natural-language rationale for high-risk transformations requires further refinement, particularly for novice learners unfamiliar with coupling metrics.

Limitations. Several limitations should be acknowledged. The Concept Extractor evaluation is restricted to 120 Python samples from a single institution, limiting generalizability across curricula and programming languages. The Refactoring component user study involves 43 students at SLIIT, constraining the broader applicability of satisfaction findings. No longitudinal study has yet been conducted to measure whether OptiCode use produces measurable improvements in student learning outcomes over time. The emotion recognition module is bounded by the known accuracy ceiling of the FER2013 dataset ($\sim 70\%$), which does not fully reflect real-world lighting and occlusion conditions. Finally, the 0.7/0.3 interview score weighting was validated on a limited internal session set and warrants larger-scale empirical validation. Addressing these limitations through multi-institutional trials, multi-language benchmarking, and longitudinal cohort studies represents the primary direction for future work.

X. CONCLUSION

This paper presented OptiCode, a four-component AI platform for computer science education. The Code Concept Extractor demonstrates that a hybrid AST+LLM pipeline achieves macro $F1 = 0.85 \pm 0.04$, outperforming LLM-only extraction by 16.4%. This improvement arises from two mechanisms: structural pre-detection providing verified concept seeds, and a post-merge step recovering LLM false

negatives. The Adaptive Weakness Detector, Code Refactoring and Risk Analyzer, and AI Interview Simulator complement the extractor to form a complete student support ecosystem. Future work will extend multi-language evaluation, expand the benchmark dataset beyond Python to multi-language corpora, and conduct longitudinal studies at SLIIT.

ACKNOWLEDGMENT

The authors acknowledge the guidance of their research supervisor and the Department of Information Technology, SLIIT. This work was conducted as part of the final-year research project (Group No. 25-26J-364) for the BSc (Hons) in Information Technology program. AI writing assistance tools were used for grammar enhancement of this manuscript in accordance with IEEE submission policy.

REFERENCES

- [1] S. Finnie-Ansley, P. Denny, B. A. Becker, A. Luxton-Reilly, and J. Prather, "The robots are coming: Exploring the implications of OpenAI Codex on introductory programming," in *Proc. Australasian Computing Education Conf. (ACE)*, 2022, pp. 10–19.
- [2] A. LeClair, S. Jiang, and C. McMillan, "A neural model for generating natural language summaries of program subroutines," in *Proc. IEEE/ACM 41st ICSE*, 2019, pp. 795–806.
- [3] X. Hu, G. Li, X. Xia, D. Lo, and Z. Jin, "Deep code comment generation," in *Proc. 26th IEEE/ACM ICPC*, 2018, pp. 200–210.
- [4] M. Chen et al., "Evaluating large language models trained on code," 2021, arXiv:2107.03374.
- automatic analysis of asynchronous video job interviews," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 573–581.
- [5] K. R. Koedinger, J. R. Anderson, W. H. Hadley, and M. A. Mark, "Intelligent tutoring goes to school in the big city," *Int. J. Artif. Intell. Educ.*, vol. 8, pp. 30–43, 1997.
- [6] Q. Liu et al., "EKT: Exercise-aware knowledge tracing for student performance prediction," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 1, pp. 100–115, 2019.
- [7] G. C. Murphy, M. Kersten, and L. Findlater, "How are Java software developers using the Eclipse IDE?" *IEEE Softw.*, vol. 23, no. 4, pp. 76–83, 2006.
- [8] I. Naim, M. I. Tanveer, D. Gildea, and M. E. Hoque, "Automated prediction and analysis of job interview performance," *IEEE Trans. Affect. Comput.*, vol. 9, no. 2, pp. 180–194, 2018.
- [9] J. Wei, R. Wang, Z. Wang, and Y. Li, "Interview question answering on knowledge graphs," in *Proc. 29th ACM Int. Conf. Multimedia*, 2021, pp. 4763–4771.
- [10] J. R. Corbett and J. R. Anderson, "Knowledge tracing: Modeling the acquisition of procedural knowledge," *User Modeling and User-Adapted Interaction*, vol. 4, no. 4, pp. 253–278, 1994.
- [11] OpenAI, "GPT-4 Technical Report," arXiv:2303.08774, 2023.
- [12] A. M. Dakhel, V. Majdinasab, A. Nikanjam, F. Khomh, M. C. Desmarais, and Z. M. J. Jiang, "GitHub Copilot AI pair programmer: Asset or liability?" *J. Syst. Softw.*, vol. 203, p. 111734, 2023.
- [13] M. Messer, M. Brown, M. Hidson, and G. Ingram, "Automated grading and feedback tools for programming education: A systematic review," *ACM Trans. Comput. Educ.*, vol. 24, no. 1, pp. 1–43, 2024.
- [14] P. Phung, D. Tyagi, A. Padala, G. Banerjee, and S. Misailovic, "Automating human tutor-style programming feedback: Leveraging GPT-4 tutor model for hints and Socratic questions," in *Proc. AAAI Symp. AI Educ.*, 2024.
- [15] S. Hemamou, G. Felhi, V. Vandenbussche, J.-C. Martin, and C. Clavel, "HireNet: A hierarchical attention model for the